Monitoring


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

Advanced Dashboarding Overview

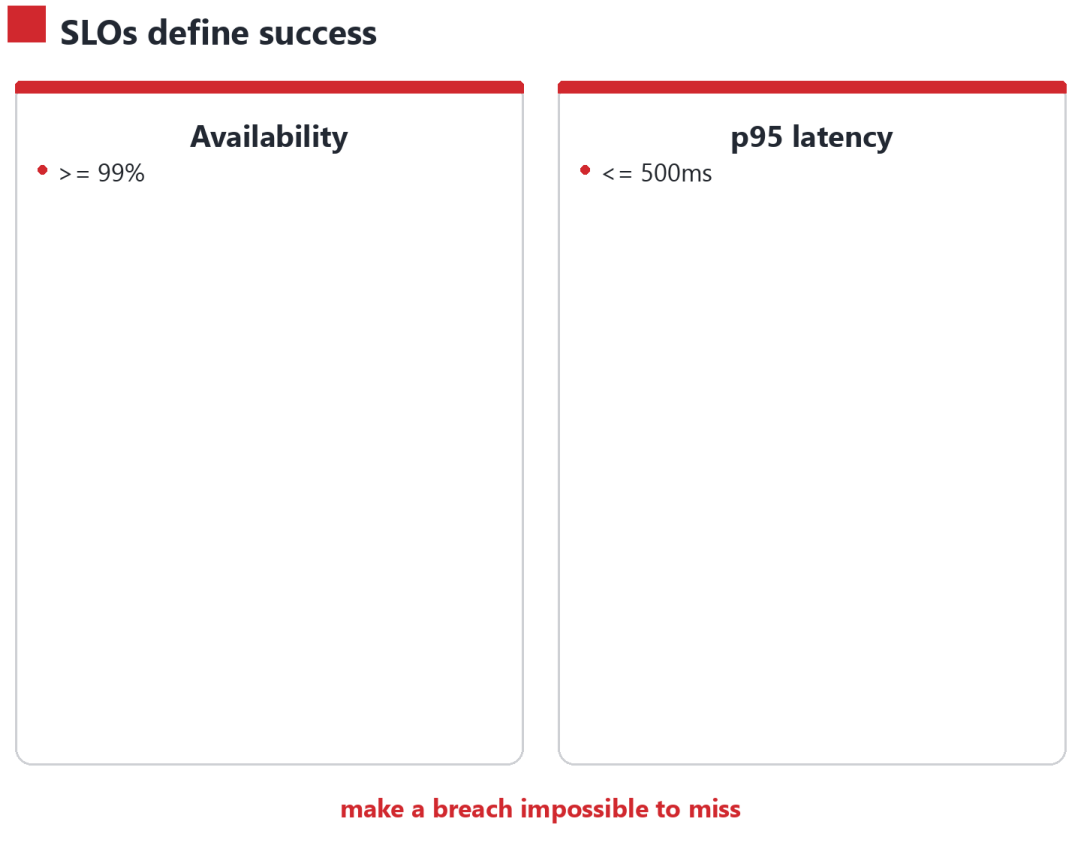
- You will build a production-grade Grafana dashboard that an on-call engineer can actually make decisions from. The goal is one view of service health, pulling real Prometheus metrics and Loki logs, not a wall of pretty but useless charts.
  - Grafana dashboards
  - Prometheus metrics
  - Loki logs
  - SLO-driven views

Service Scenario

- Your team owns an internal API that many other systems depend on, so when it struggles the blast radius is large. During an incident the dashboard has to answer is it healthy in seconds, because everyone downstream is waiting.
  - Shared dependency
  - High impact
  - On-call usage
  - Operational focus

Service Level Objectives

- An SLO defines what good actually means for this service, like 99 percent availability and a p95 latency under 500ms. The dashboard's job is to make a breach impossible to miss, so on-call sees it before the customer complaint arrives.
  - Availability ≥ 99%
  - p95 latency ≤ 500ms
  - Measured continuously
  - Visible in Grafana

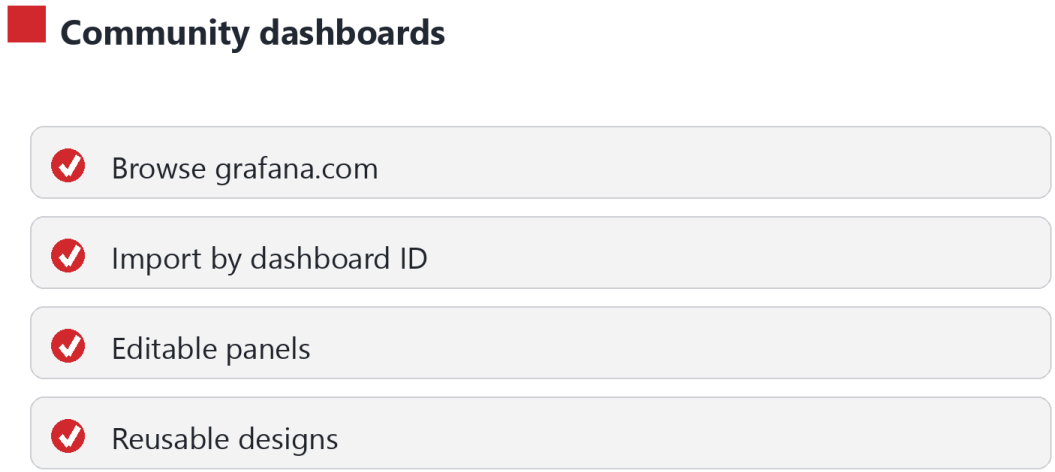


Operational Questions

- Every panel should earn its place by answering a real operational question: is it up, is it fast, is it scaling, can I debug it. Charts that do not answer one of those just add noise that slows you down at 2am.
  - Is it up?
  - Is it fast?
  - Is it scaling?
  - Can I debug it?

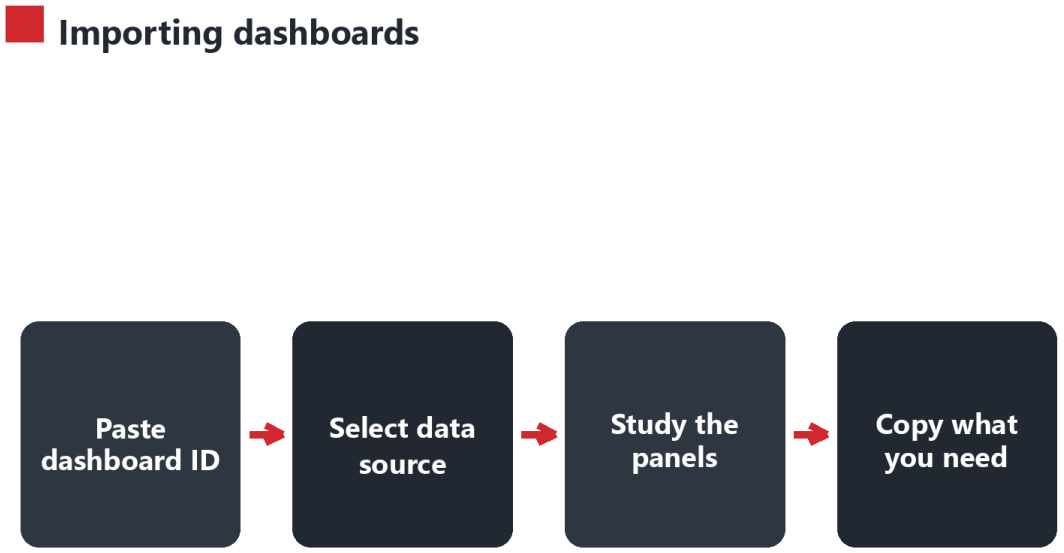
Community Dashboards

- You rarely need to start from a blank page. Grafana's community library has thousands of prebuilt dashboards you can import by ID, which are great for borrowing panel ideas and proven query patterns.
  - grafana.com
  - Import by ID
  - Editable panels
  - Reusable designs



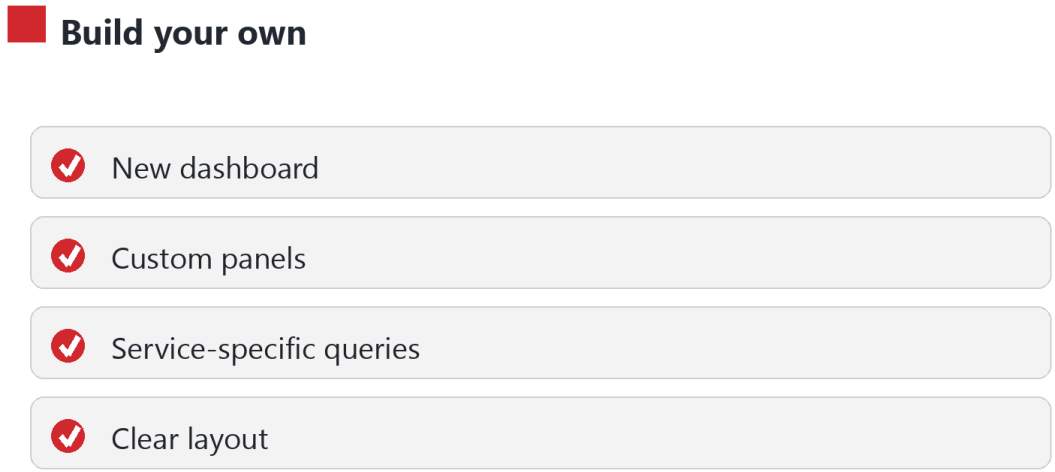
Importing Dashboards

- An imported dashboard is a starting point, not the finished product. The useful move is to import one, study how its panels are built, then copy the panels you want into a dashboard tailored to your own service.
  - Paste dashboard ID
  - Select data source
  - Explore queries
  - Reuse panels



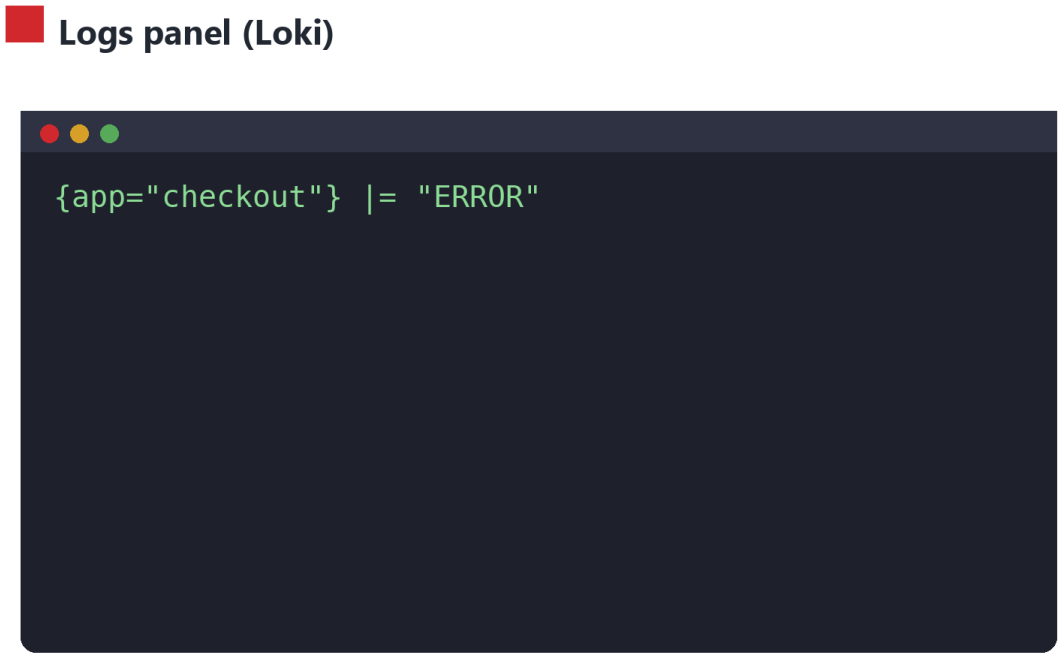
Creating a New Dashboard

- Building your own dashboard is what guarantees the panels match your SLOs, not someone else's. You start blank and add service-specific queries, so every panel maps to a question your team actually asks.
  - New dashboard
  - Custom panels
  - Service-specific queries
  - Clear layout



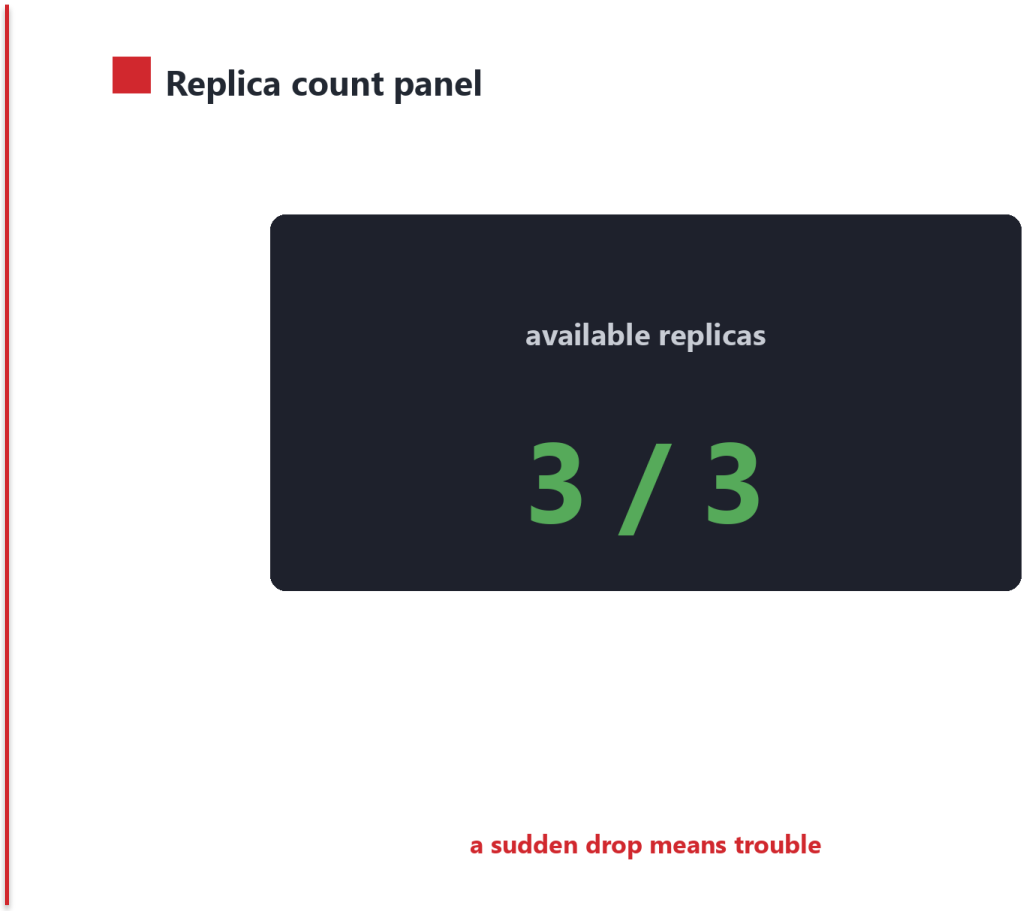
Logs Panel

- Metrics tell you something is wrong; logs tell you what. A logs panel wired to Loki, with a search variable, lets you jump from a spiking error graph straight to the matching log lines without leaving the dashboard.
  - Logs panel
  - Search variable
  - Filter by app
  - Time correlation



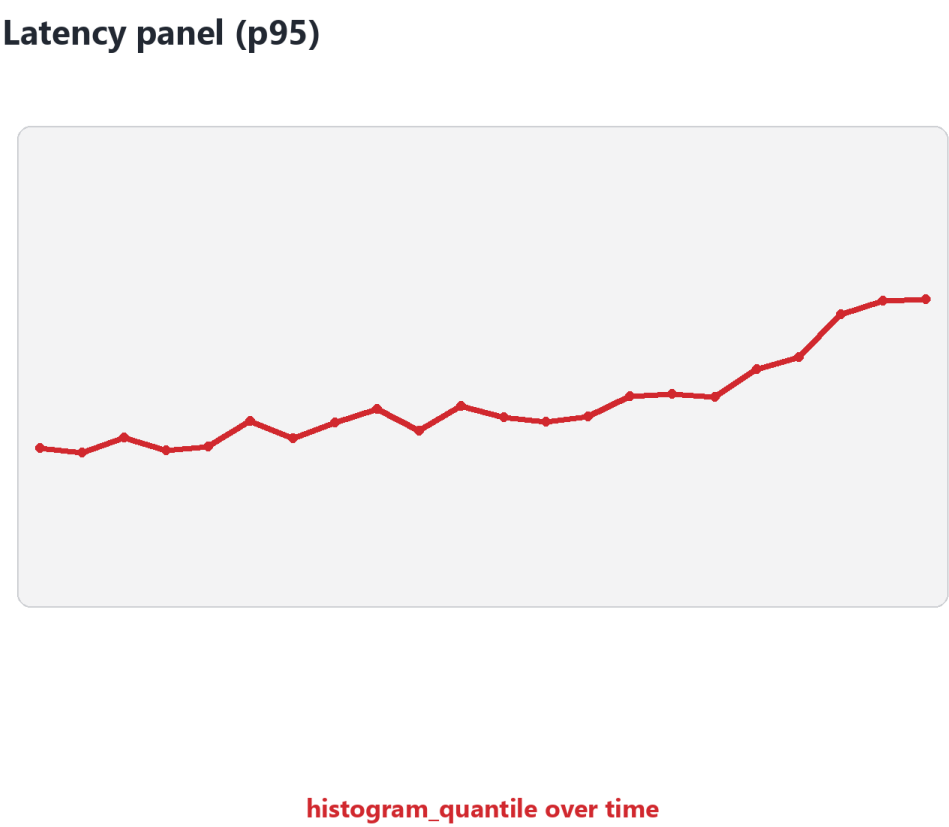
Replica Count Panel

- Replica count is a quick read on whether the service is scaled and stable. When available replicas suddenly drop below expected, that panel is often the first visible sign of crashes or a failed rollout.
  - Stat or Gauge
  - Available replicas
  - Expected vs actual
  - Autoscaling insight



Latency Panel

- Latency is the clearest proxy for how fast the service feels to users. Graphing p95 with histogram_quantile shows the worst-case experience over time, which is what an SLO and your users actually care about.
  - Time series
  - p95 latency
  - histogram_quantile
  - Trend analysis

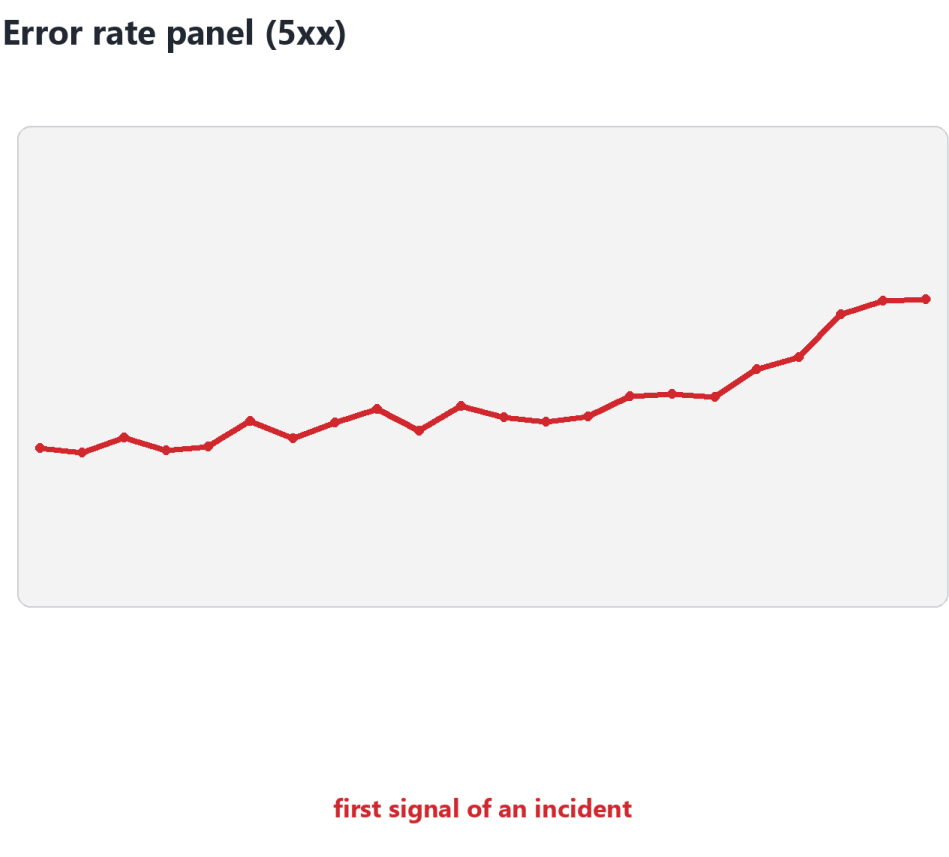


Availability Panel

- Availability is the headline SLO number: the share of requests that succeed. A single stat panel showing success rate, with a red threshold at your target, tells on-call instantly whether you are in or out of budget.
  - Success ratio
  - Exclude 5xx
  - Stat panel
  - Thresholds

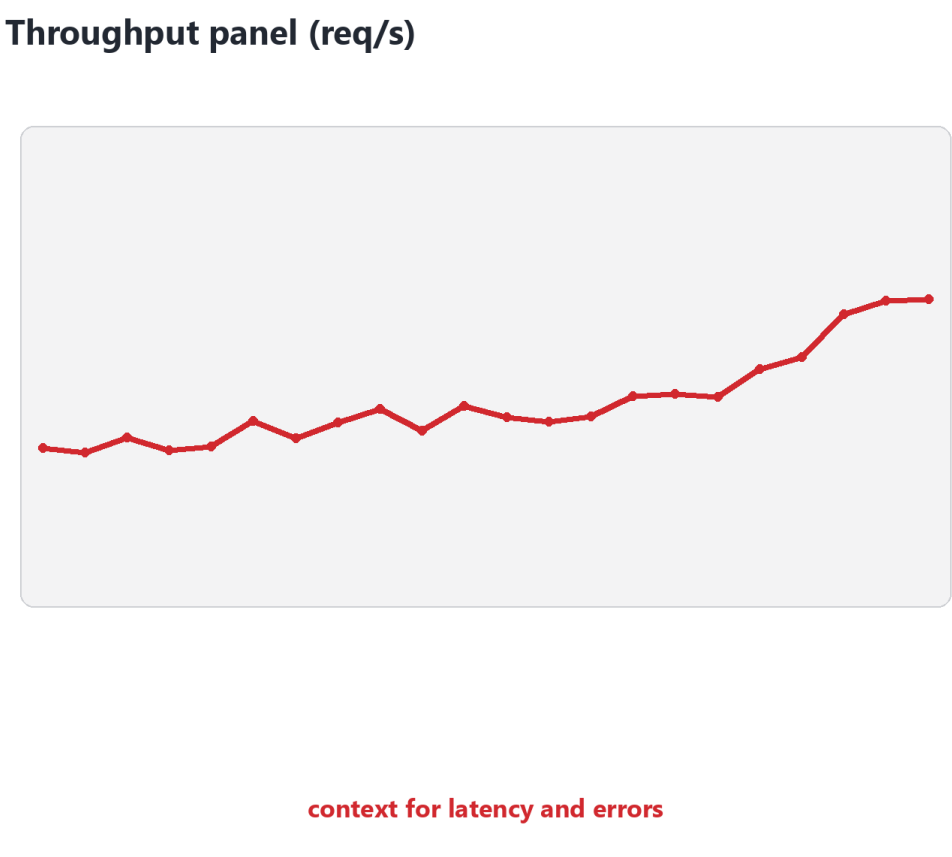
Error Rate Panel

- Tracking errors on their own, separate from total traffic, makes failures jump out early. A rising rate of 5xx responses is usually the first signal of an incident and a natural thing to alert on.
  - 5xx responses
  - Rate over time
  - Alert candidate
  - Incident signal



Throughput Panel

- Throughput, requests per second, shows how much load the service is under. It gives context for the other panels, because a latency spike during a traffic surge tells a very different story than one at normal load.
  - Requests per second
  - Traffic spikes
  - Correlation
  - Capacity insight



Dashboard Variables

- Variables turn one dashboard into many. A namespace or deployment variable lets the same panels serve staging and production, so you maintain one dashboard instead of copy-pasting it per environment.
  - Namespace variable
  - Deployment variable
  - Log filters
  - Dynamic queries

Dashboard variables

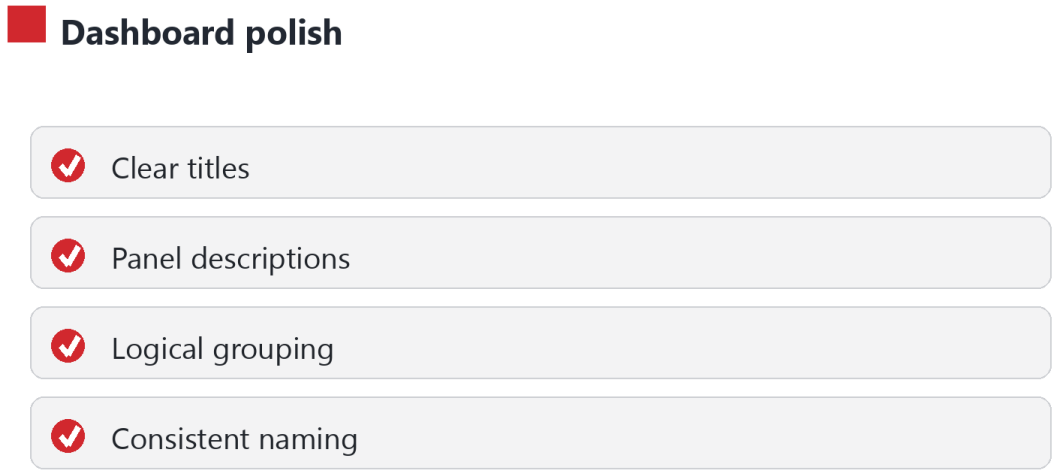
```
$namespace = production
$deployment = checkout

one dashboard, many targets
```

Metrics and Logs Together

- The fastest investigations start in metrics and end in logs. You spot the anomaly on a metric panel, then jump to logs for the same time window, and the alignment between them is what reveals the root cause.
  - Start with metrics
  - Jump to logs
  - Time alignment
  - Root cause

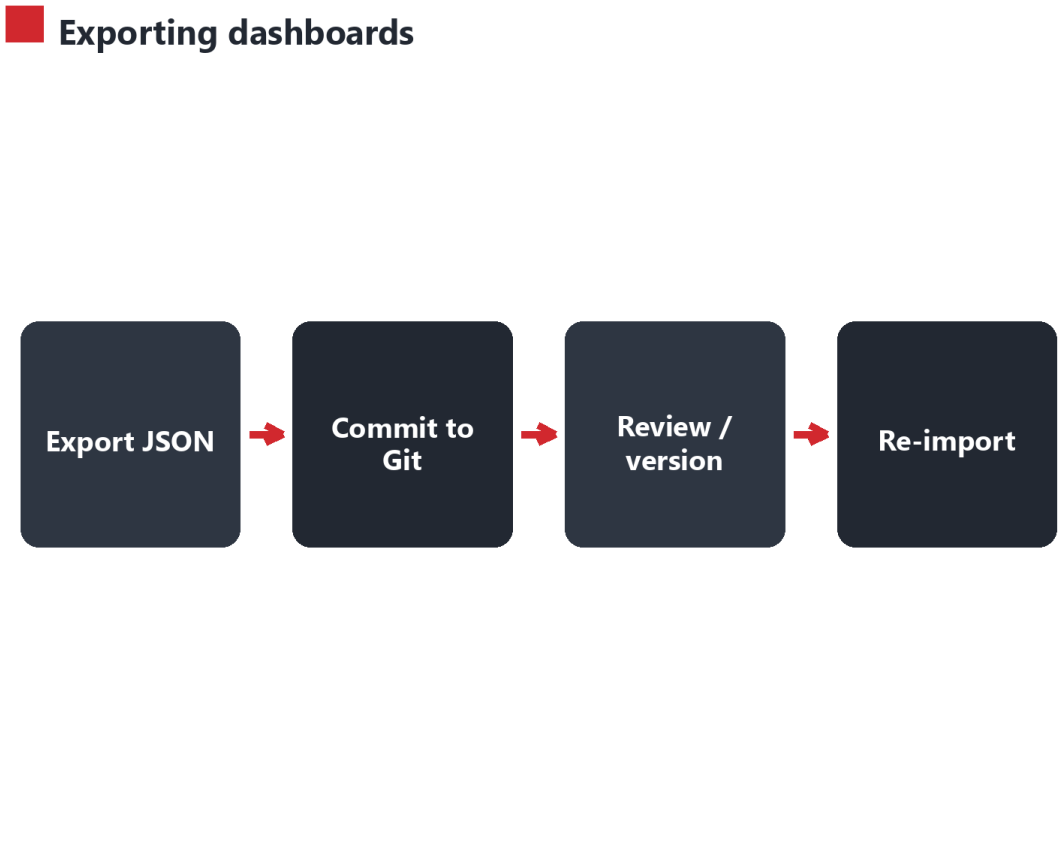
Dashboard Polish

- A dashboard is a shared artifact the whole team reads under pressure, so clarity is not optional. Clear titles, descriptions, grouping, and consistent naming let a teammate use your dashboard correctly at 3am without a tour.
  - Titles
  - Descriptions
  - Panel grouping
  - Consistent naming



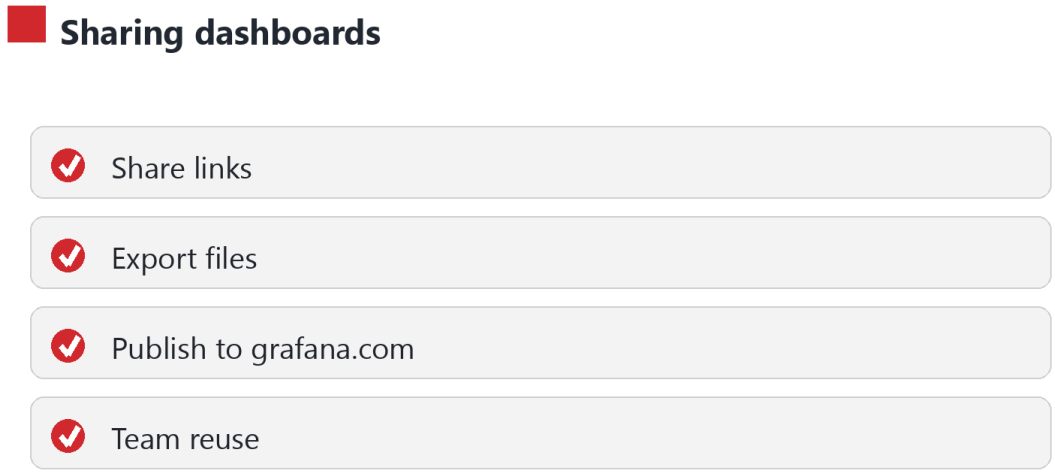
Exporting Dashboards

- A dashboard is just JSON, which means it can live in Git like any other code. Exporting and committing it gives you version history, review, and the same dashboard reliably re-imported anywhere.
  - Export JSON
  - Store in Git
  - Re-import
  - Share easily



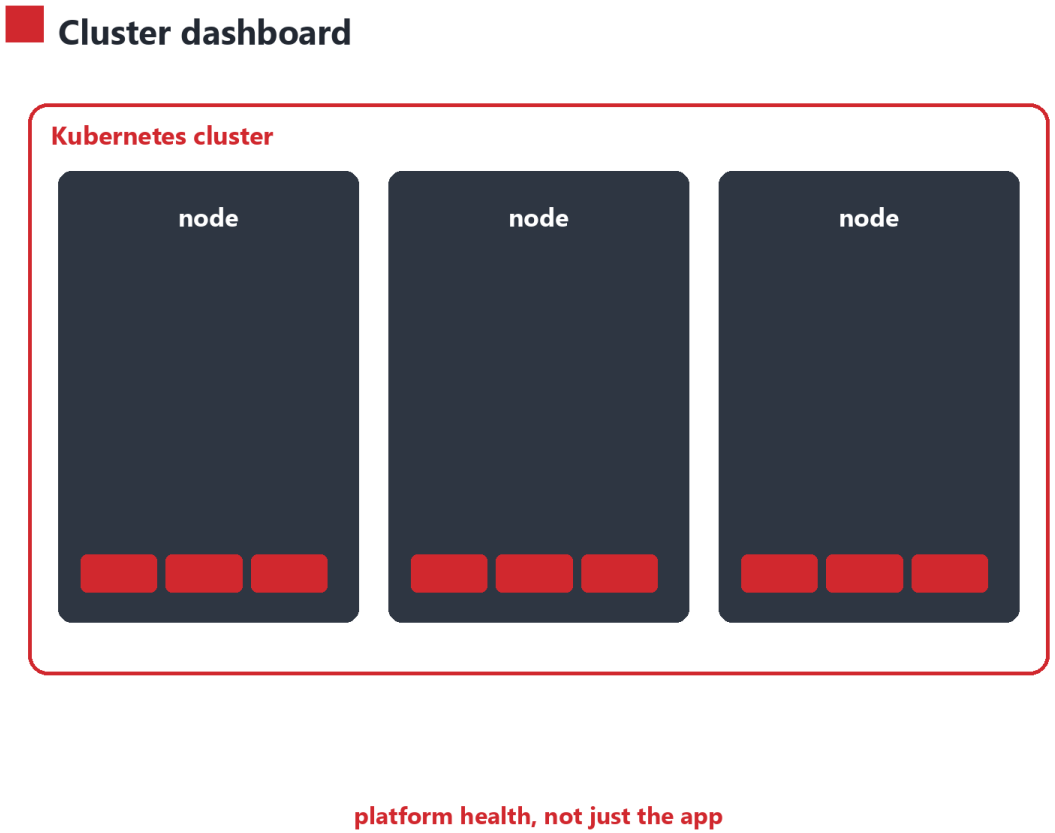
Sharing Dashboards

- Good dashboards are worth standardizing across the team. Sharing them by link, exported file, or the Grafana library means everyone investigates the same way, instead of each engineer reinventing the same panels.
  - Share links
  - Export files
  - grafana.com
  - Team reuse



Cluster Dashboard

- Application dashboards do not tell the whole story; sometimes the platform is the problem. A cluster dashboard showing node metrics, pod counts, and capacity is what lets you tell a code issue from an infrastructure one.
  - Node metrics
  - Pod counts
  - Infra errors
  - Capacity overview



On-Call Workflow

- Under pressure, a dashboard should guide a repeatable workflow, not require improvisation. Check availability, inspect latency, review errors, then debug via logs is the path that turns a panic into a procedure.
  - Check availability
  - Inspect latency
  - Review errors
  - Debug via logs

MCQ 1 – Purpose of Dashboard

- What is the main goal of the dashboard?
  - A. Store logs
  - B. Replace alerts
  - C. Evaluate service health
  - D. Monitor only infra

MCQ 1 – Purpose of Dashboard – Answer

- What is the main goal of the dashboard?
  - A. Store logs
  - B. Replace alerts
  - **C. Evaluate service health**
  - D. Monitor only infra

MCQ 2 – SLO

- What does an SLO represent?
  - A. Alert rule
  - B. Business target
  - C. Log format
  - D. Metric type

MCQ 2 – SLO – Answer

- What does an SLO represent?
  - A. Alert rule
  - **B. Business target**
  - C. Log format
  - D. Metric type

MCQ 3 – p95

- What does p95 latency show?
  - A. Average
  - B. Fastest
  - C. Worst 5%
  - D. Total

MCQ 3 – p95 – Answer

- What does p95 latency show?
  - A. Average
  - B. Fastest
  - **C. Worst 5%**
  - D. Total

MCQ 4 – Logs

- Why use logs?
  - A. Replace metrics
  - B. Explain failures
  - C. Store dashboards
  - D. Generate alerts

MCQ 4 – Logs – Answer

- Why use logs?
  - A. Replace metrics
  - **B. Explain failures**
  - C. Store dashboards
  - D. Generate alerts

MCQ 5 – Variables

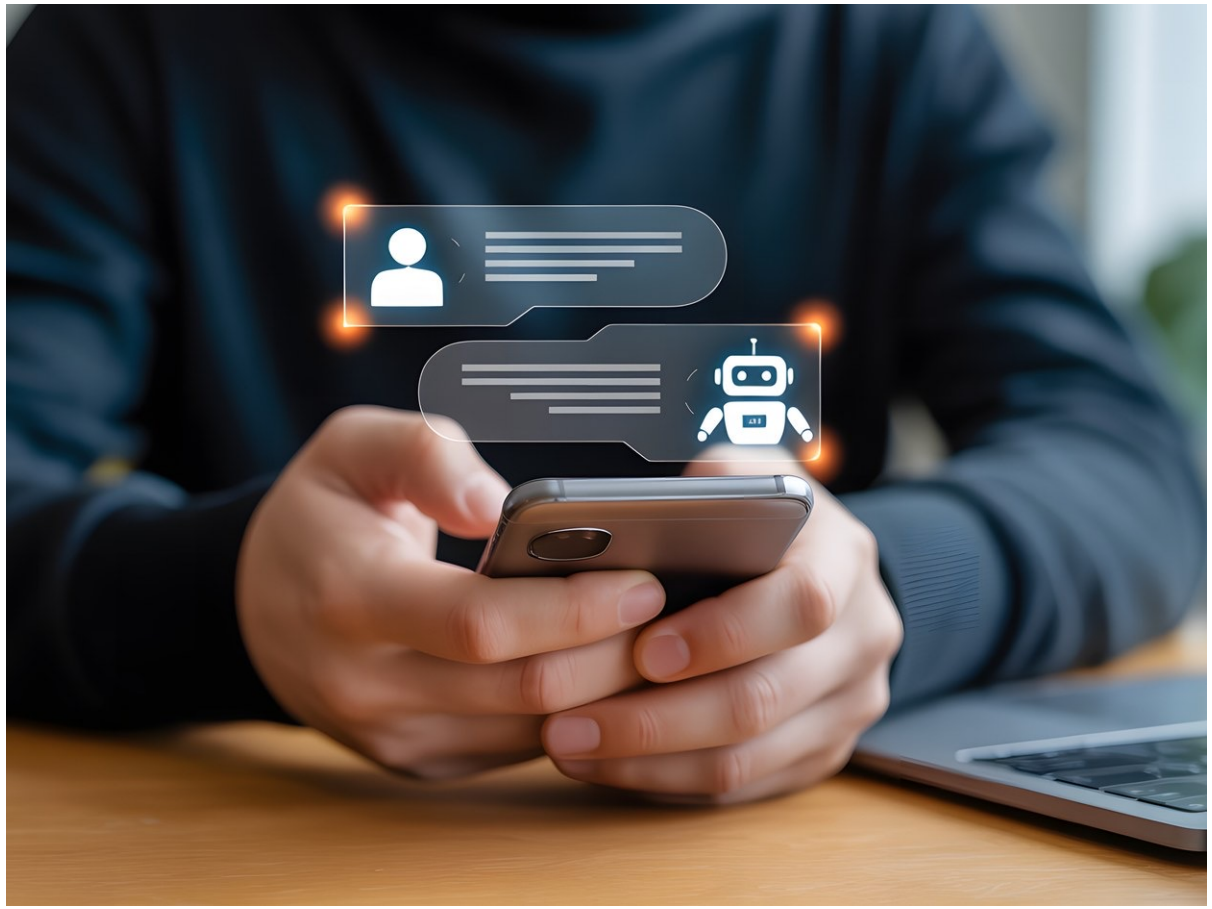
- Why use variables?
  - A. Faster queries
  - B. Reduce cost
  - C. Reusable dashboards
  - D. Replace PromQL

MCQ 5 – Variables – Answer

- Why use variables?
  - A. Faster queries
  - B. Reduce cost
  - **C. Reusable dashboards**
  - D. Replace PromQL

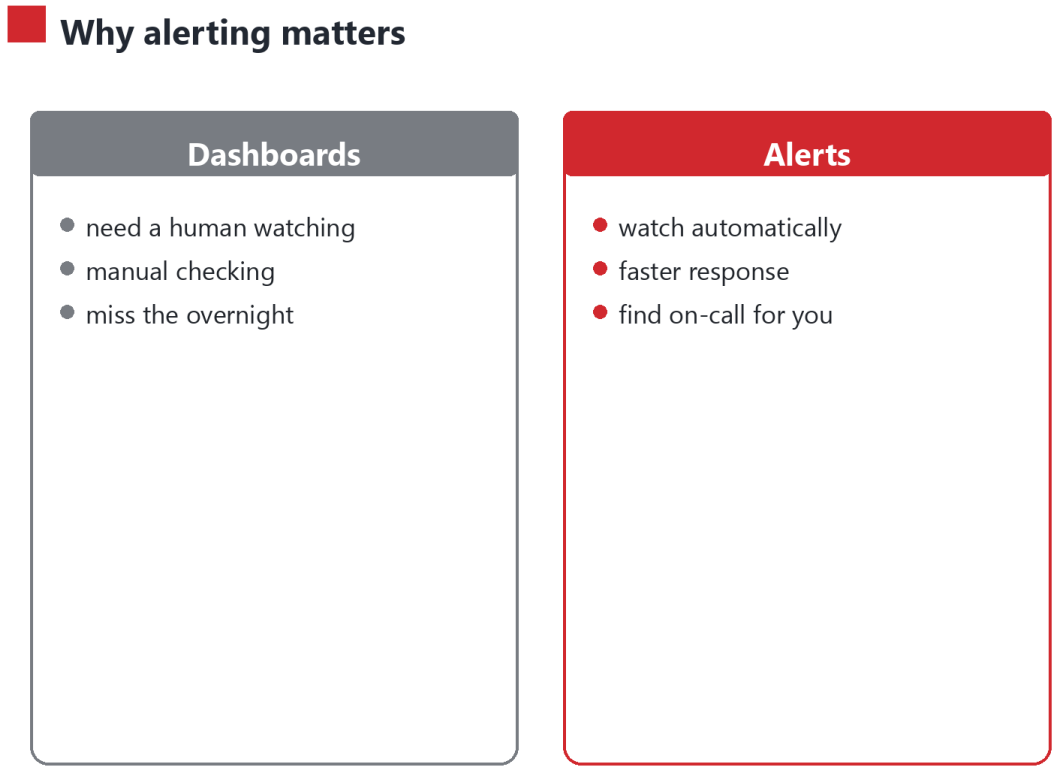
Alerting Overview

- Dashboards only help when someone is looking; alerts watch for you. Grafana alerting evaluates your Prometheus metrics and pushes a Slack message the moment a condition is met, so problems find the on-call engineer.
  - Grafana alerting
  - Prometheus metrics
  - Slack notifications
  - On-call workflows



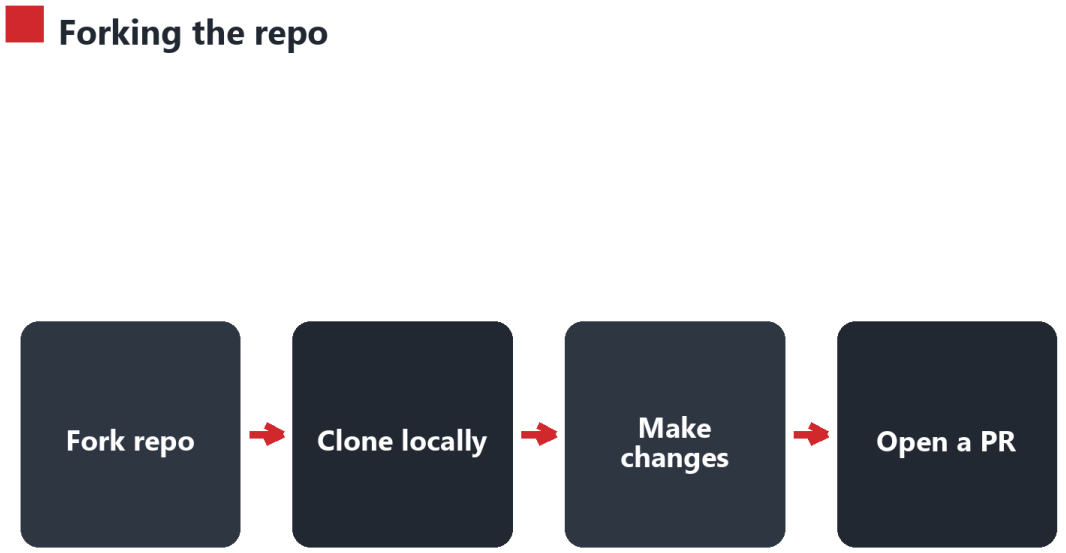
Why Alerting Matters

- Relying on humans to keep checking dashboards does not scale and fails overnight. Alerting automates detection, which is what shrinks the time between a failure starting and someone actually responding to it.
  - Faster response
  - Reduced downtime
  - Clear signals
  - Operational readiness



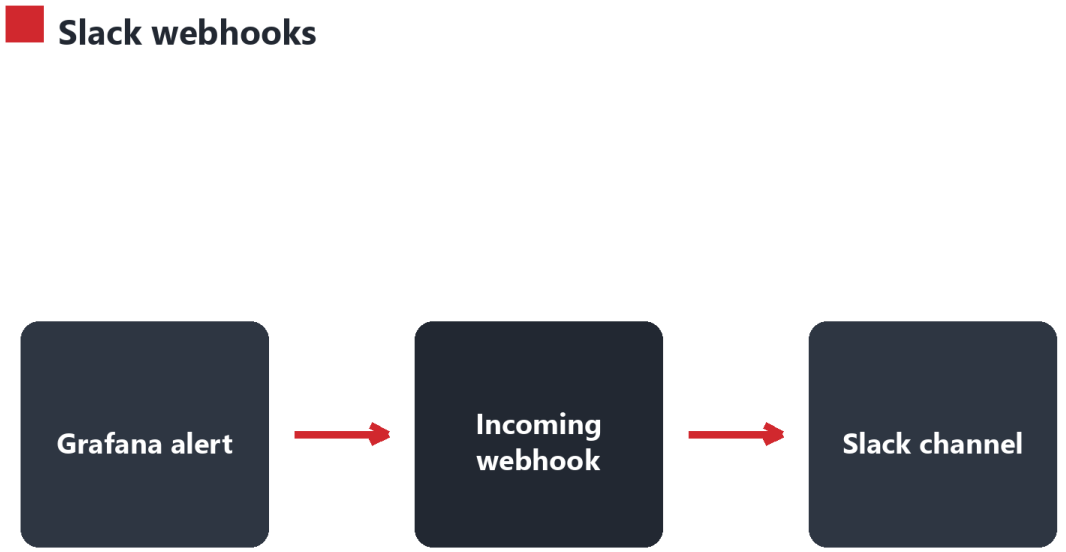
Forking the Repo

- You will do this lab in your own fork so you can break things safely. Because everything flows through Git, every change you make is tracked and easy to undo, which is the whole point of a GitOps workflow.
  - Fork repo
  - Clone locally
  - Safe experimentation
  - GitOps flow



Slack Webhooks

- Slack is where most teams already live during an incident, so that is where alerts should land. An incoming webhook gives Grafana a secure URL to post to, turning a firing rule into a message in the right channel.
  - Slack app
  - Incoming webhook
  - Channel selection
  - Secure URL



MySQL Availability Metric

- The alert in this lab is driven by a single, honest signal: mysql_up, which is 1 when the database is reachable and 0 when it is not. A binary metric like this is the simplest possible basis for a meaningful alert.
  - 1 = up
  - 0 = down
  - Prometheus metric
  - Binary signal

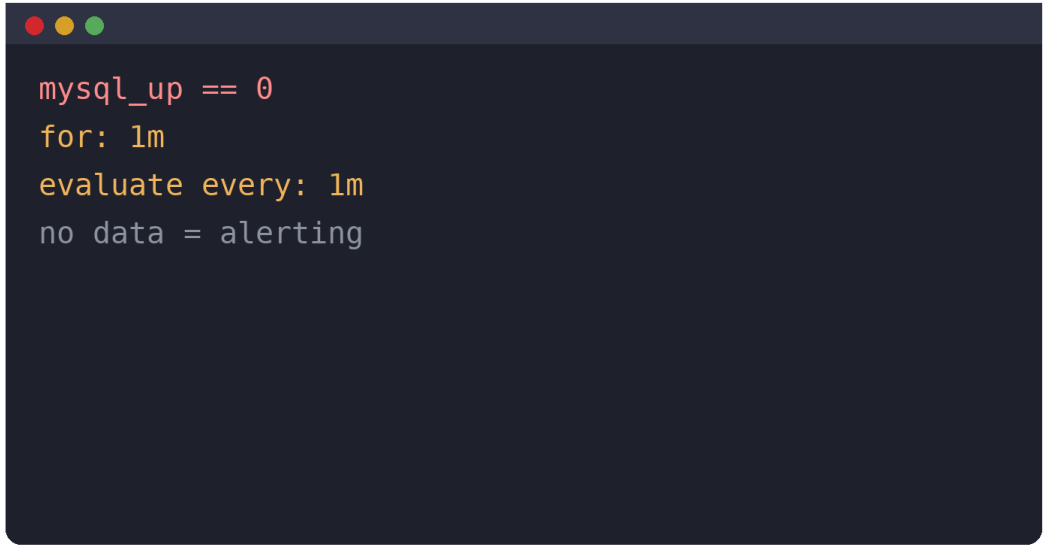
MySQL availability metric

```
mysql_up == 1   -> up
mysql_up == 0   -> down
```

Creating an Alert Rule

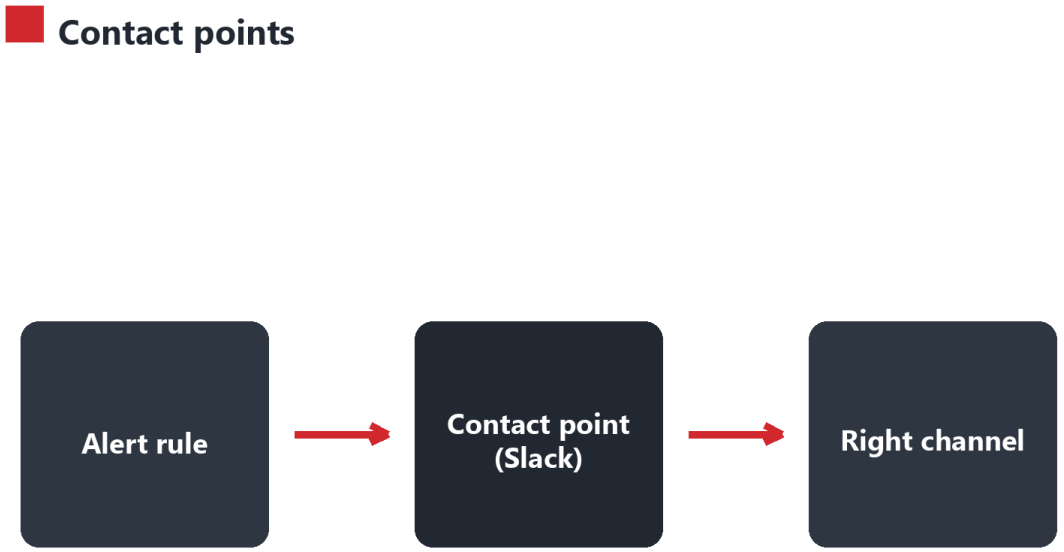
- An alert rule turns a query into a condition Grafana watches for you. Here mysql_up == 0, evaluated every minute and required to hold for a minute, with no-data treated as a problem, so a truly down database reliably fires.
  - mysql_up == 0
  - Evaluate every minute
  - For 1 minute
  - Treat no data as alert

Creating an alert rule



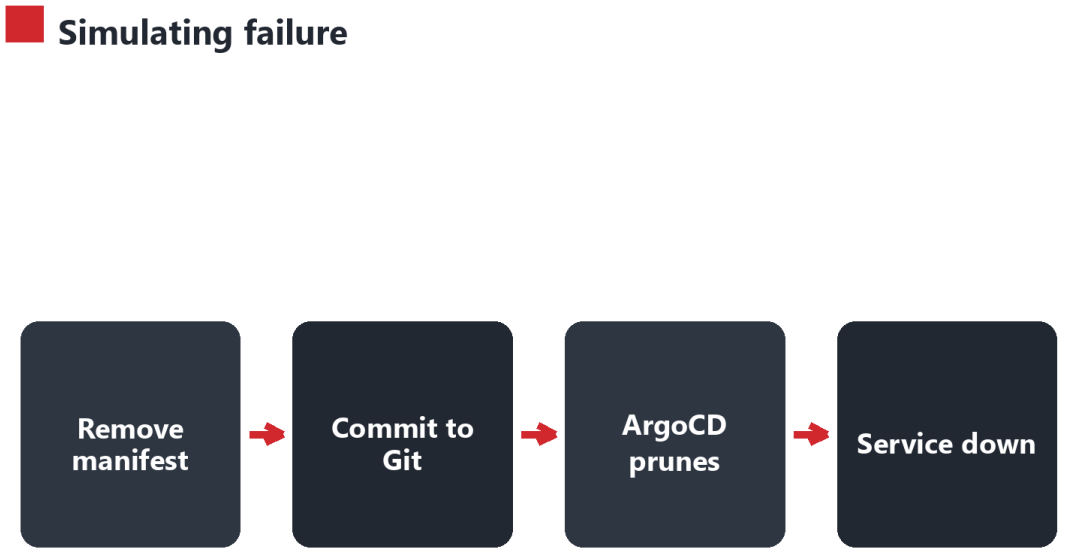
Contact Points

- A rule decides when to alert; a contact point decides where the alert goes. You define a Slack contact point with your webhook URL once, then attach it to rules so the right people are notified.
  - Add contact point
  - Slack type
  - Webhook URL
  - Attach to rule



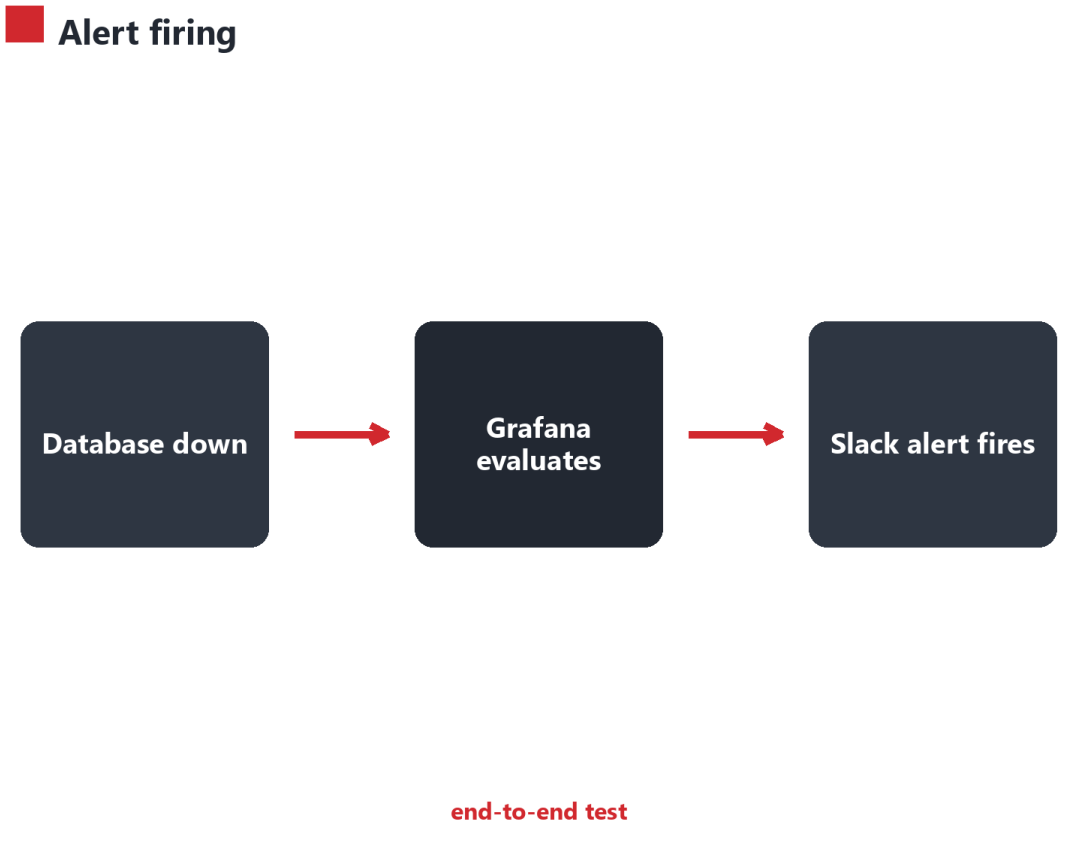
Simulating Failure

- To prove the alert works, you cause a real outage through Git. Removing the deployment and committing it lets ArgoCD prune the resource, which takes the database down exactly the way a bad change would in production.
  - Remove deployment
  - Commit change
  - ArgoCD prunes
  - Failure occurs



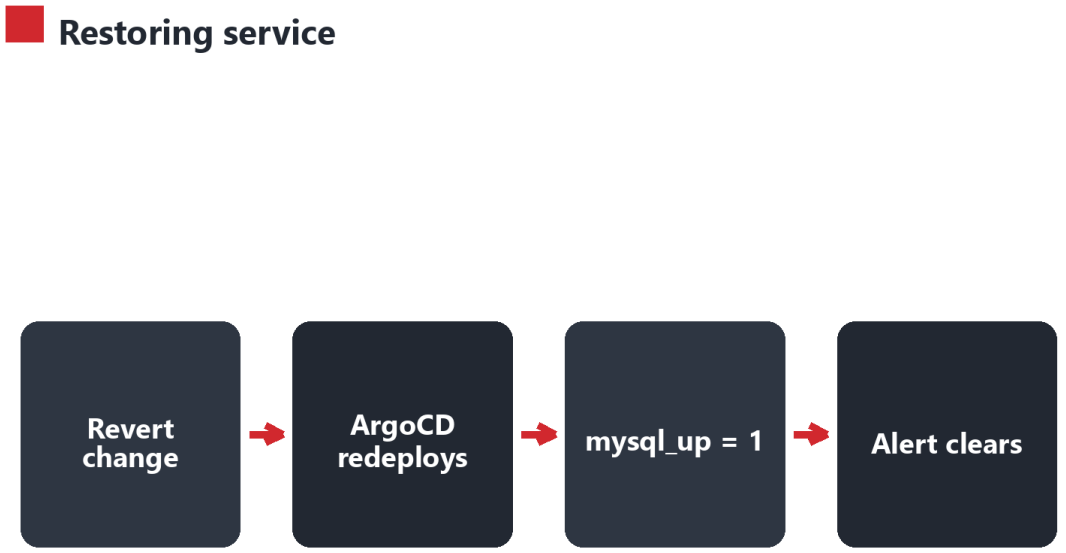
Alert Firing

- This is the end-to-end test: the database is gone, Grafana evaluates the rule, and a message lands in Slack. Seeing that full path fire is what gives you confidence the alert will work during a real incident.
  - Slack message
  - Rule validation
  - End-to-end test
  - Incident flow



Restoring Service

- Recovery runs the same loop in reverse. You revert the Git change, ArgoCD redeploys the database, mysql_up returns to 1, and the alert resolves on its own, which is how a healthy alerting system should behave.
  - Revert change
  - ArgoCD redeploys
  - mysql_up returns
  - Alert clears



MCQ 1 – Alerting

- What is alerting for?
  - A. Visualize logs
  - B. Notify engineers
  - C. Store metrics
  - D. Replace dashboards

MCQ 1 – Alerting – Answer

- What is alerting for?
  - A. Visualize logs
  - **B. Notify engineers**
  - C. Store metrics
  - D. Replace dashboards

MCQ 2 – Webhook

- Slack webhook does what?
  - A. Scrapes metrics
  - B. Sends alerts
  - C. Deploys apps
  - D. Stores logs

MCQ 2 – Webhook – Answer

- Slack webhook does what?
  - A. Scrapes metrics
  - **B. Sends alerts**
  - C. Deploys apps
  - D. Stores logs

MCQ 3 – mysql_up

- mysql_up value 0 means?
  - A. Slow
  - B. Down
  - C. Healthy
  - D. Restart

MCQ 3 – mysql_up – Answer

- mysql_up value 0 means?
  - A. Slow
  - **B. Down**
  - C. Healthy
  - D. Restart

MCQ 4 – ArgoCD

- Removing manifest does what?
  - A. Nothing
  - B. Deletes pod
  - C. Prunes resource
  - D. Stops Prometheus

MCQ 4 – ArgoCD – Answer

- Removing manifest does what?
  - A. Nothing
  - B. Deletes pod
  - **C. Prunes resource**
  - D. Stops Prometheus

MCQ 5 – Resolve

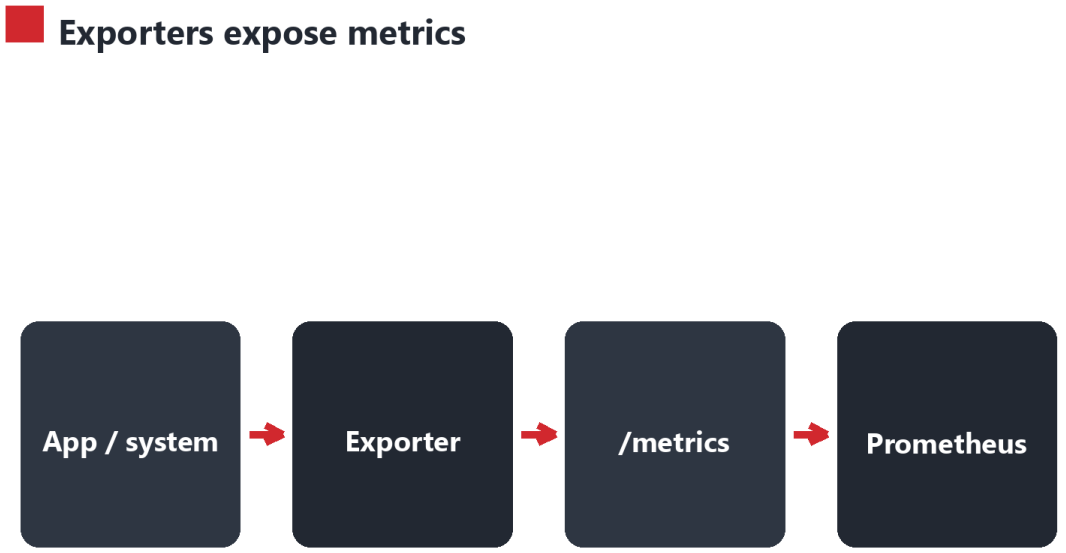
- When does alert resolve?
  - A. Slack restart
  - B. Grafana restart
  - C. MySQL restored
  - D. Prometheus restart

MCQ 5 – Resolve – Answer

- When does alert resolve?
  - A. Slack restart
  - B. Grafana restart
  - **C. MySQL restored**
  - D. Prometheus restart

Exporters Overview

- Plenty of systems you need to monitor do not speak Prometheus natively. An exporter is a small process that sits beside them, reads their internal stats, and republishes them at a /metrics endpoint Prometheus can scrape.
  - Standalone processes
  - Expose /metrics
  - Scraped by Prometheus
  - Operational overhead



Why Exporters Exist

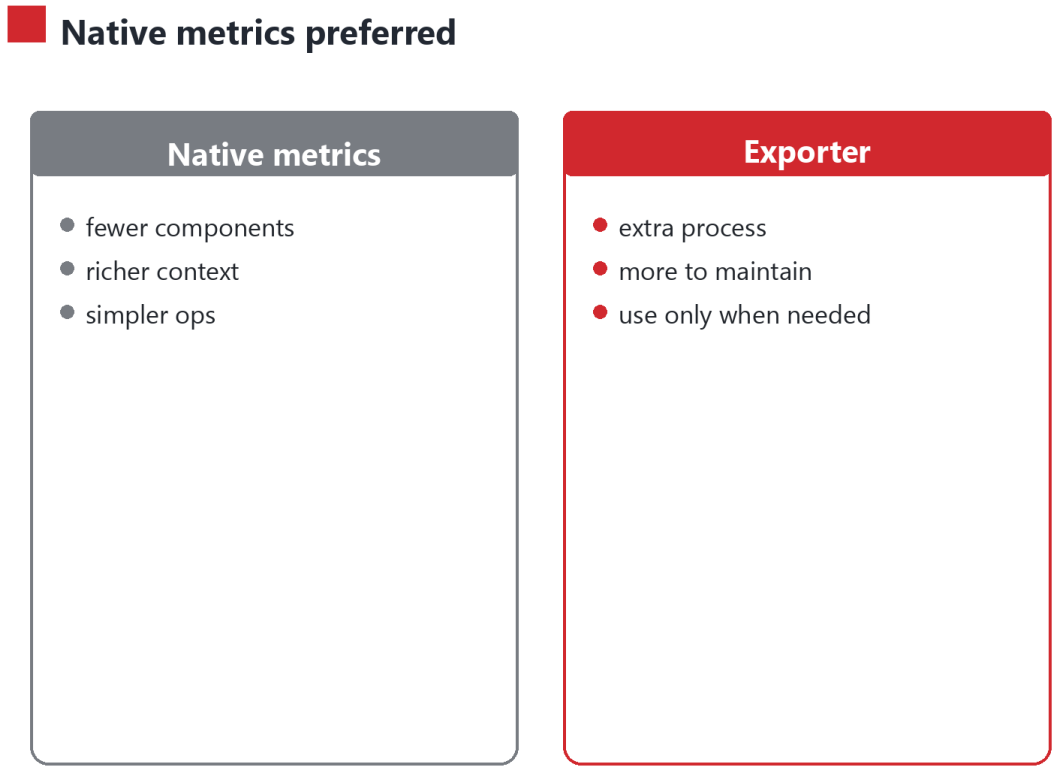
- Databases, caches, and web servers like MySQL, Redis, and Nginx rarely expose Prometheus metrics on their own. Exporters fill that gap, bringing legacy and third-party systems into the same monitoring stack as your apps.
  - MySQL
  - Redis
  - Nginx
  - Legacy systems

Exporter Tradeoffs

- Every exporter is another process to deploy, secure, version, and keep running. That extra surface area is real operational cost, and an exporter that dies silently becomes a blind spot of its own.
  - Extra deployment
  - Versioning
  - Security
  - Failure points

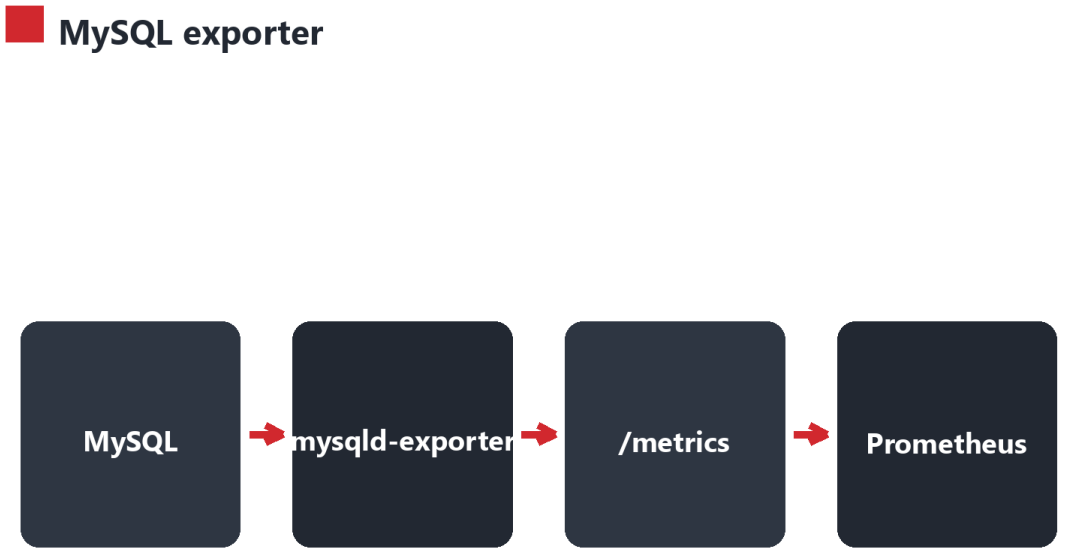
Native Metrics Preferred

- When an app can expose Prometheus metrics itself, that is almost always the better choice. Native metrics mean fewer moving parts and richer context, so reach for an exporter only when there is no native option.
  - Fewer components
  - Better context
  - Lower latency
  - Simpler ops



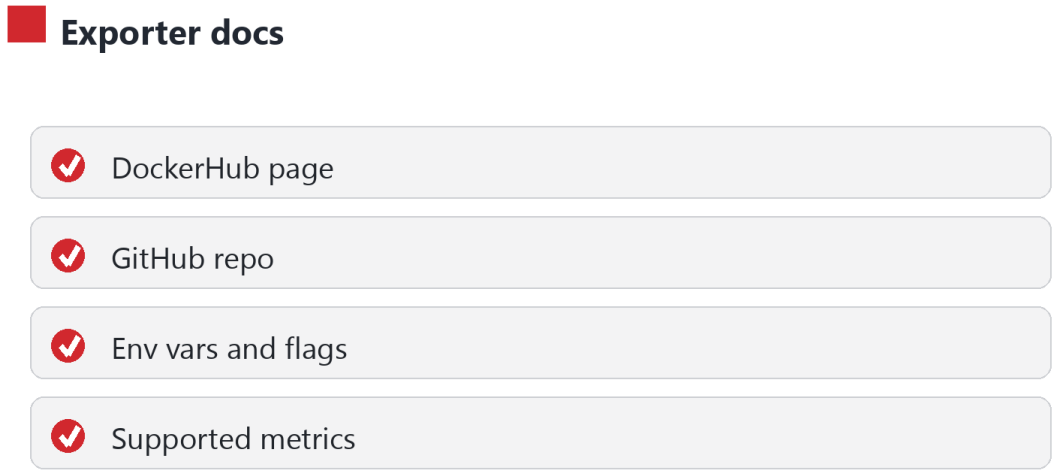
MySQL Exporter

- The MySQL exporter, mysqld-exporter, connects to the database, runs internal queries, and exposes the results in Prometheus format. It runs as its own service so Prometheus can scrape database health like any other target.
  - mysqld-exporter
  - Queries internals
  - Prometheus format
  - Metrics endpoint



Exporter Docs

- Before deploying an exporter, the docs are your map. The DockerHub and GitHub pages list the environment variables, flags, and exactly which metrics it exposes, which saves you from guessing at configuration.
  - DockerHub
  - GitHub
  - Env vars
  - Supported metrics



Testing Exporters

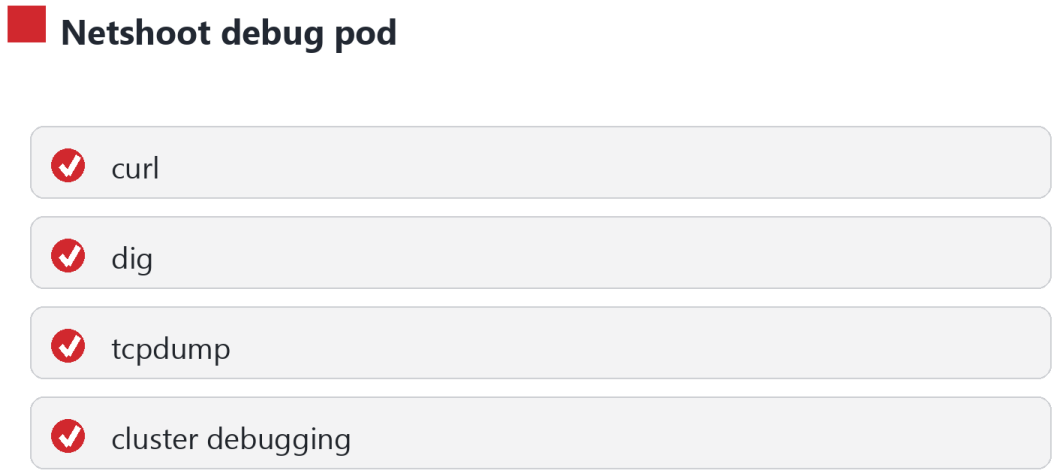
- Never assume an exporter works; confirm it. The quick check is to curl its /metrics endpoint from inside the cluster and look at the output, so you catch a misconfiguration before Prometheus silently scrapes nothing.
  - Find pod
  - Use netshoot
  - Curl /metrics
  - Inspect output

Testing exporters

```
kubectl exec netshoot --
  curl mysqld-exporter:9104/metrics
```

Netshoot Pod

- Netshoot is a throwaway debug container packed with network tools like curl, dig, and tcpdump. Dropping one into the cluster gives you a place to test connectivity and endpoints without touching your app pods.
  - curl
  - dig
  - tcpdump
  - Cluster debugging



ServiceMonitor

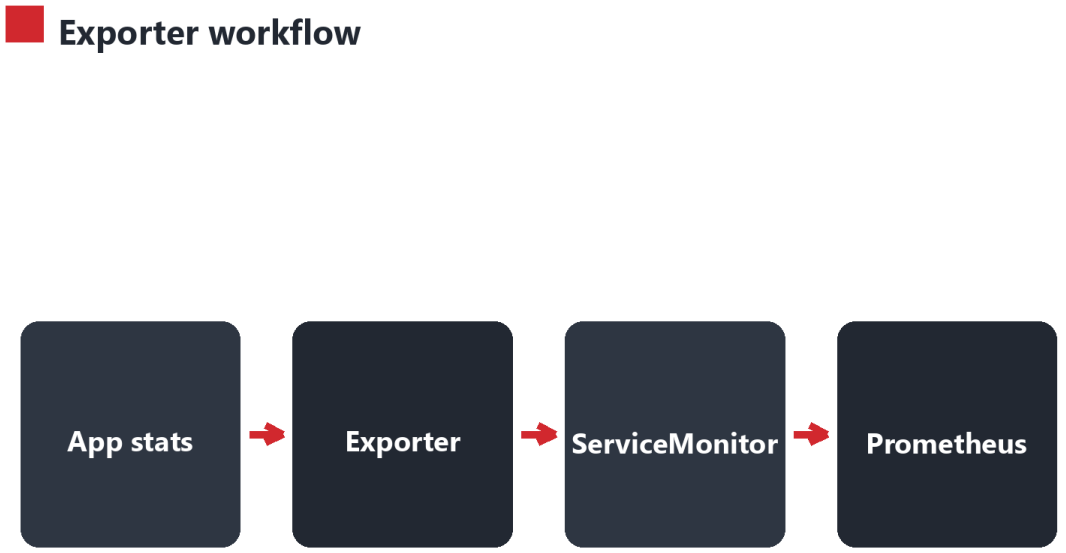
- A ServiceMonitor is the Kubernetes CRD that tells Prometheus what to scrape, declaratively. Using label selectors and a scrape interval, it keeps scrape config in Git instead of a hand-edited file, which fits GitOps.
  - Label selectors
  - Scrape interval
  - Declarative config
  - GitOps-friendly

ServiceMonitor (CRD)

```
kind: ServiceMonitor
selector:  app=mysql-exporter
interval: 30s
```

Exporter Workflow

- An exporter is just another metrics endpoint once it is wired in. App stats flow into the exporter, a ServiceMonitor points Prometheus at it, and from then on it behaves like any other scrape target.
  - App stats
  - Exporter
  - ServiceMonitor
  - Prometheus



MCQ 1 – Exporters

- Purpose of exporters?
  - A. Replace Prometheus
  - B. Expose metrics
  - C. Store logs
  - D. Trigger alerts

MCQ 1 – Exporters – Answer

- Purpose of exporters?
  - A. Replace Prometheus
  - **B. Expose metrics**
  - C. Store logs
  - D. Trigger alerts

MCQ 2 – Endpoint

- Metrics exposed at?
  - A. /health
  - B. /status
  - C. /metrics
  - D. /debug

MCQ 2 – Endpoint – Answer

- Metrics exposed at?
  - A. /health
  - B. /status
  - **C. /metrics**
  - D. /debug

MCQ 3 – MySQL

- MySQL exporter does?
  - A. Backup DB
  - B. Run queries
  - C. Expose metrics
  - D. Manage users

MCQ 3 – MySQL – Answer

- MySQL exporter does?
  - A. Backup DB
  - B. Run queries
  - **C. Expose metrics**
  - D. Manage users

MCQ 4 – ServiceMonitor

- ServiceMonitor role?
  - A. Deploy exporters
  - B. Define scraping
  - C. Visualize data
  - D. Store logs

MCQ 4 – ServiceMonitor – Answer

- ServiceMonitor role?
  - A. Deploy exporters
  - **B. Define scraping**
  - C. Visualize data
  - D. Store logs

MCQ 5 – Best Practice

- When use exporters?
  - A. Always
  - B. Only DBs
  - C. When no native metrics
  - D. Only prod

MCQ 5 – Best Practice – Answer

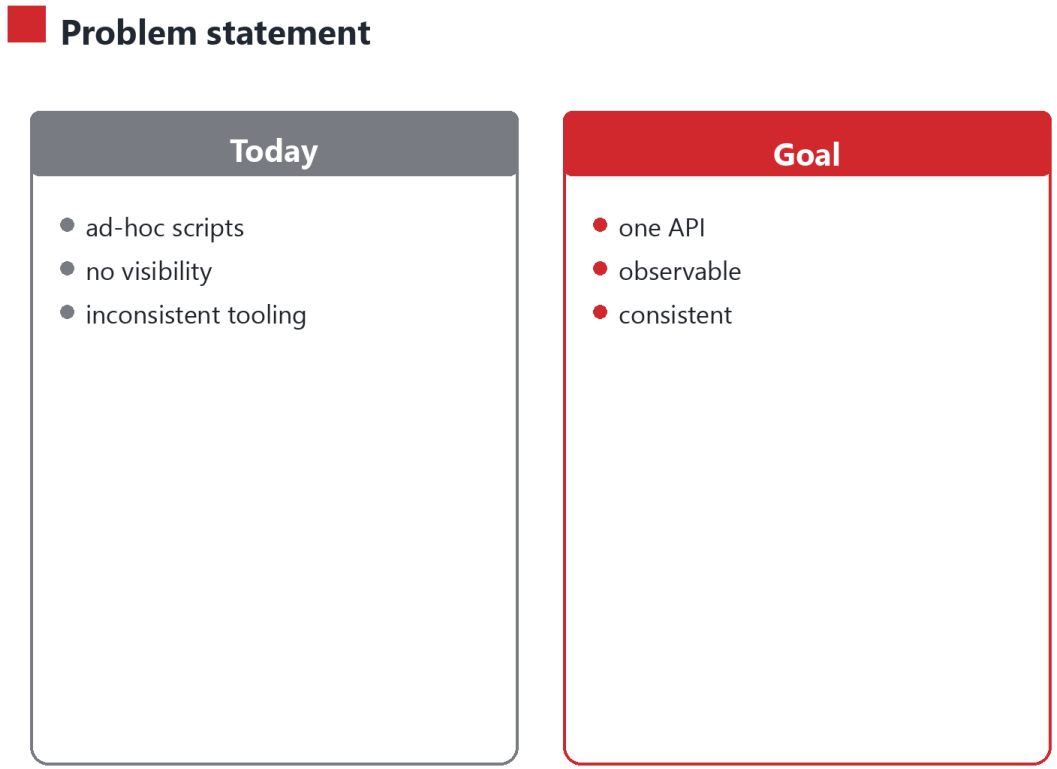
- When use exporters?
  - A. Always
  - B. Only DBs
  - **C. When no native metrics**
  - D. Only prod

Final Lab Overview

- For the capstone you will build a centralized Automation Gateway API, a small platform service. It exposes its own business metrics and ships through GitOps, tying together everything from the previous labs.
  - Automation API
  - Business metrics
  - Argo CD
  - Grafana proof

Problem Statement

- Automation in most orgs is a pile of ad-hoc scripts with no visibility and no consistency. That fragmentation is an operational risk, and centralizing it behind one observable API is what this lab sets out to fix.
  - Ad-hoc scripts
  - No visibility
  - Inconsistent tooling
  - Operational risk

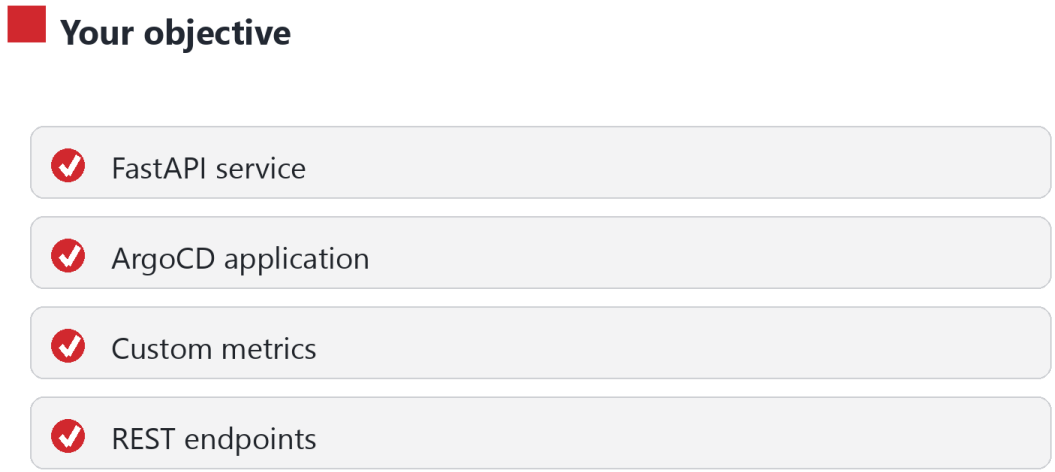


Given Environment

- You are not starting from scratch: the EKS cluster, the LGTM stack, a reference API, and ArgoCD are already in place. Your job is to build on that foundation, the way you would join an existing platform team.
  - EKS
  - LGTM stack
  - Reference API
  - Argo CD ready

Your Objective

- Your task is to create the automation-gateway service, deploy it through ArgoCD, and prove it is observable. That means a FastAPI service with real REST endpoints and custom metrics, managed declaratively in Git.
  - FastAPI service
  - Argo CD app
  - Custom metrics
  - REST endpoints



API Endpoints

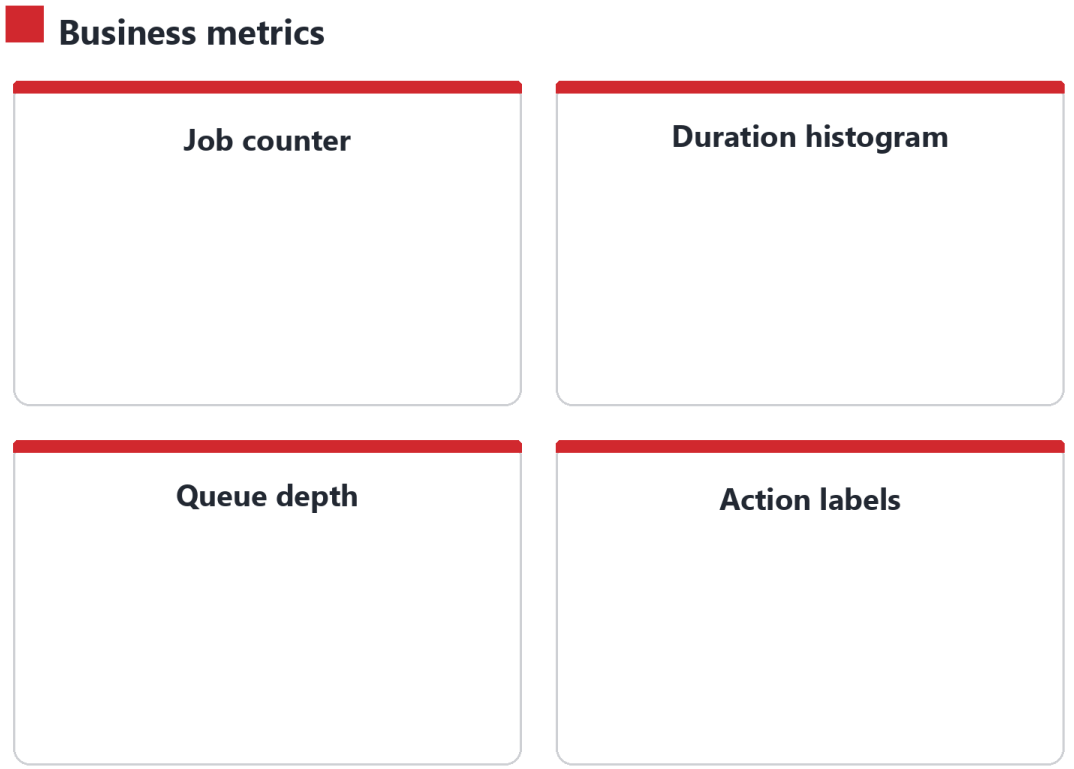
- The API exposes a small, clear surface: a health check, endpoints to create and read jobs, and a /metrics endpoint. Jobs run asynchronously, which is what makes queue depth and duration worth measuring.
  - /healthz
  - /v1/jobs
  - /v1/jobs/{id}
  - /metrics

API endpoints

```
GET  /healthz
POST /v1/jobs
GET  /v1/jobs/{id}
GET  /metrics
```

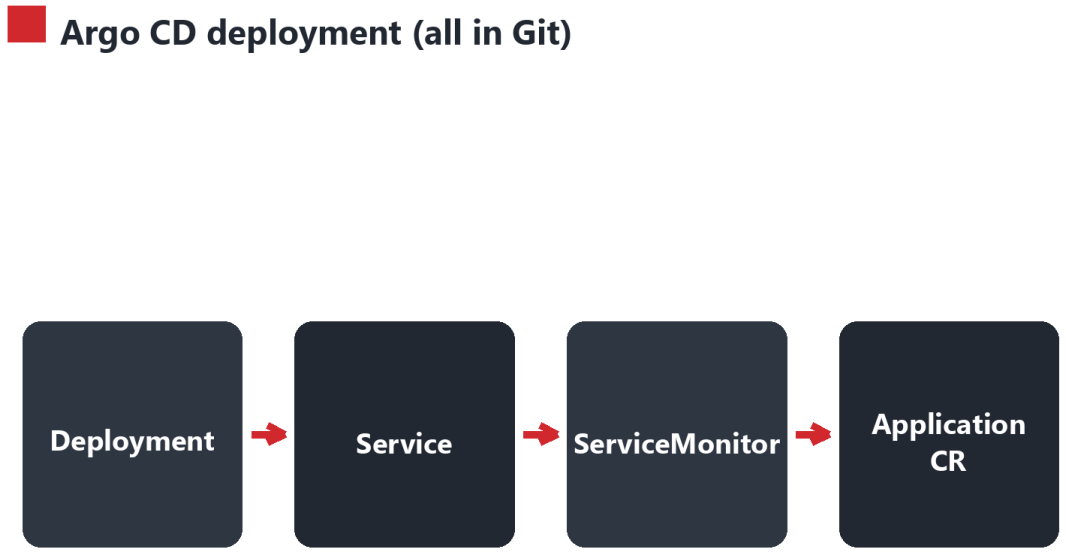
Business Metrics

- The interesting metrics here describe outcomes, not infrastructure. A counter of jobs, a histogram of how long they take, and a queue-depth gauge, labeled by action, tell you whether the automation is doing its work.
  - Job counter
  - Duration histogram
  - Queue depth
  - Action labels



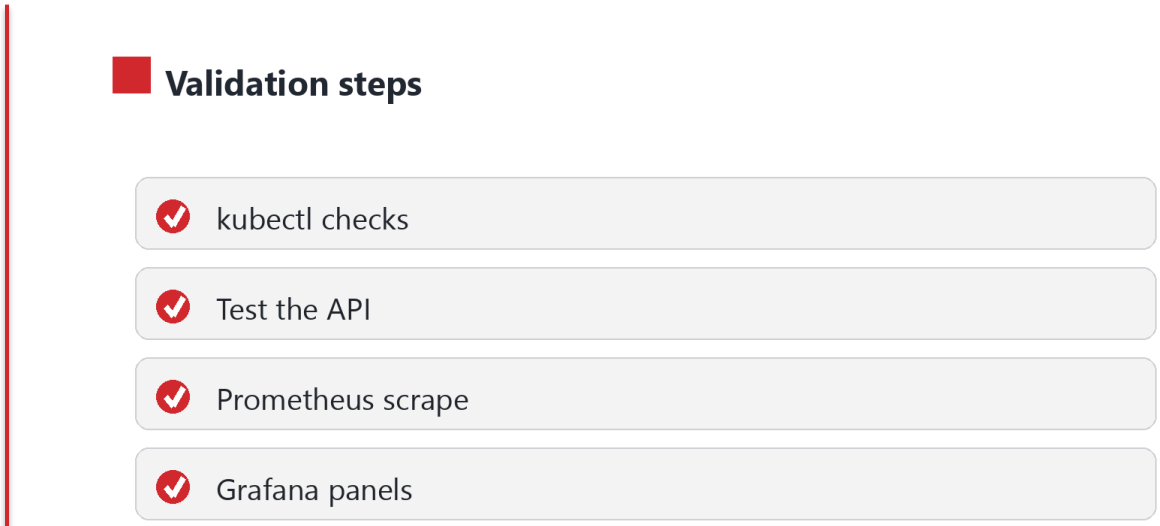
Argo CD Deployment

- Everything ships through Git using the app-of-apps pattern. The Deployment, Service, ServiceMonitor, and ArgoCD Application all live as manifests, so the whole service is reproducible and auditable from the repo.
  - Deployment
  - Service
  - ServiceMonitor
  - Application CR



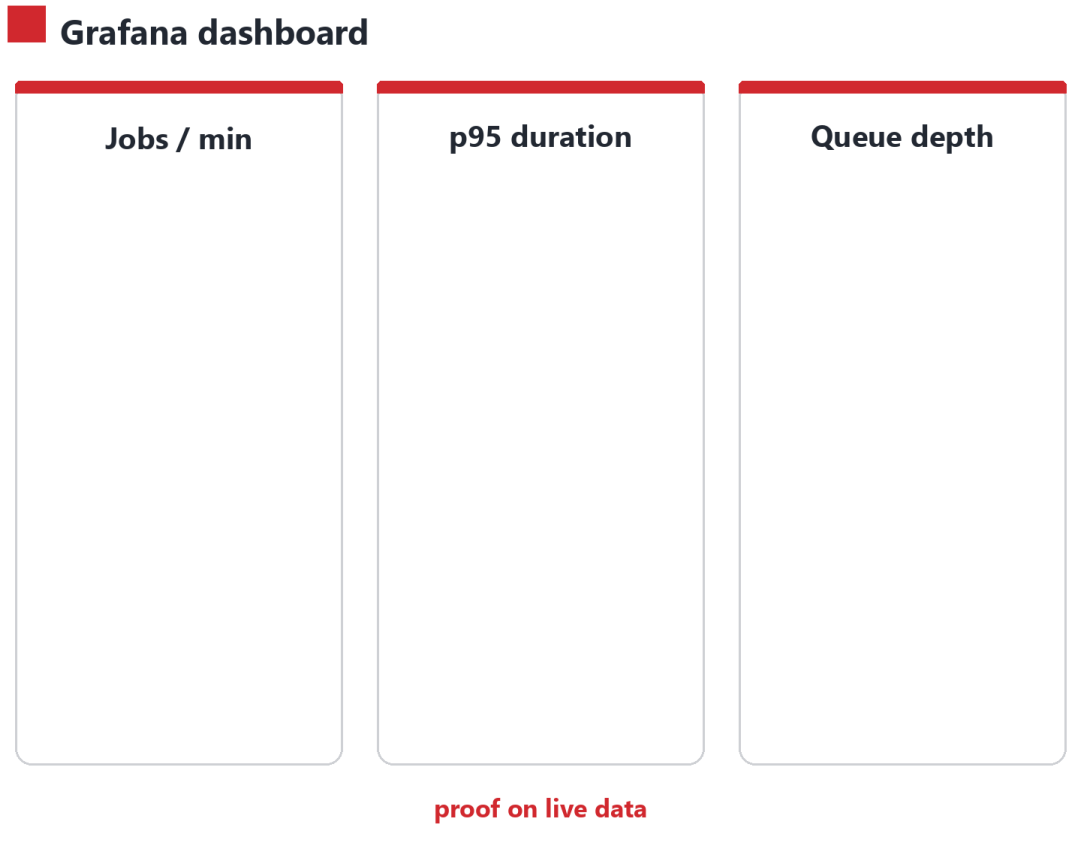
Validation Steps

- Done means proven end to end, not just deployed. You check the workload with kubectl, hit the API, confirm Prometheus is scraping it, and see the data show up in Grafana panels.
  - kubectl checks
  - Test API
  - Prometheus scrape
  - Grafana panels



Grafana Dashboard

- The final proof is a small dashboard built from your business metrics. Jobs per minute, p95 duration, and queue depth on live data is what demonstrates the automation is observable, not just running.
  - Jobs per minute
  - p95 duration
  - Queue depth
  - Live data



Why This Matters

- What you have built is a platform control plane, a stable API in front of changeable tooling. Because the interface stays put while the tools behind it can be swapped, the automation stays observable and production-ready.
  - Stable interfaces
  - Replaceable tools
  - Observable automation
  - Production-ready